PROJECT HANDBOOK

SalesGenie AI

A detailed guide to the four-module lead-management application: what each file does, how the pieces connect, how to run it, and what the current implementation can and cannot do.

Project: salesgenie_four_modules

Application modules: Lead Management, Lead Intelligence, AI Outreach, and Lead Scoring

Technology: Python, Streamlit, FastAPI, Pydantic, SQLite

> **Reading this guide:** It describes the code currently present in the project. Where the README or environment example differs from the implementation, those differences are called out rather than silently treated as working features.

Prepared from the project source, configuration, README, and sample CSV. The existing SQLite database is explained by its schema; its stored lead records are not reproduced here.

1. Project at a glance

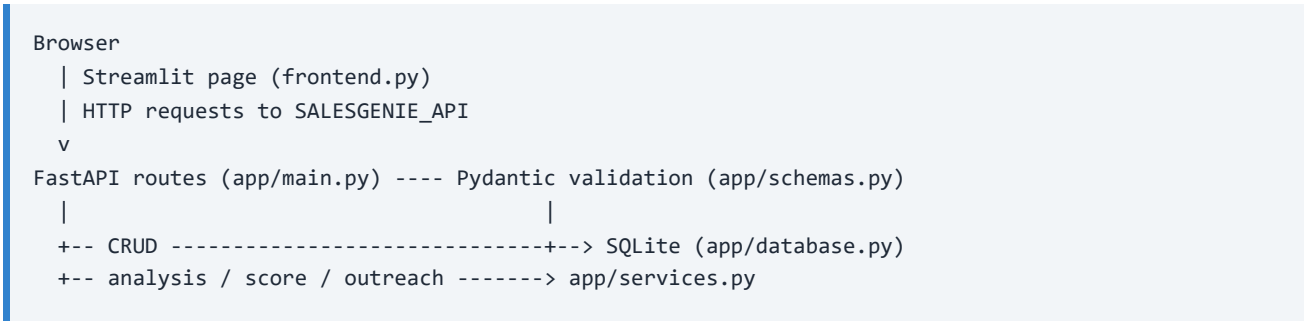
SalesGenie AI is a small local web application for recording prospects and using their recorded details to produce lightweight sales guidance. A browser-based Streamlit interface sends HTTP requests to a FastAPI backend. The backend validates input, reads or writes a SQLite database, and calls helper functions for company hints, lead scores, and outreach drafts.

The four modules are connected through the same lead record. A user first creates or updates a lead. The intelligence, outreach, and scoring pages then select that lead and request a result from the API. Analysis is based on fields already entered by a user; this project does not independently research companies or verify contact information.

> **Important:** "AI" in the product name does not mean every feature is a trained AI system. The scoring and company analysis are deterministic rules. Outreach uses a local template by default and also contains an optional, currently inconsistent Groq API branch described in Section 9.

Architecture and request flow

1. **Streamlit UI** in `frontend.py` displays forms, selectors, tables, and results.
2. **HTTP API** in `app/main.py` exposes endpoints and validates request bodies with Pydantic models.
3. **Business logic** in `app/services.py` calculates company hints and scores, and composes outreach drafts.
4. **SQLite storage** in `app/database.py` persists lead records in `data/salesgenie.db`.



2. Project files and folders

Path	Purpose	What to know
`frontend.py`	Streamlit web interface	Provides four pages, calls the API using Requests, shows results, and reports connection failures.
`app/main.py`	FastAPI application and routes	Initializes the database at startup, defines health and lead endpoints, and delegates business logic.
`app/schemas.py`	Request validation models	Defines fields and validation limits for creating/updating leads and requesting outreach.
`app/services.py`	Analysis, score, and outreach functions	Industry hints and scoring are rules, not models trained on historical outcomes.

Path	Purpose	What to know
`app/database.py`	SQLite setup and row conversion	Creates the leads table if absent, configures row access, and converts stored integer flags to booleans.
`app/__init__.py`	Python package marker	Empty file; allows imports such as `app.main`.
`data/salesgenie.db`	SQLite database file	Persistent local data. The schema is initialized at backend startup; avoid sharing this file if it contains real contact data.
`data/sample_leads.csv`	Three example lead rows	Illustrates the expected CSV columns. There is no CSV import feature in the current UI or API.
`requirements.txt`	Python dependency constraints	Declares FastAPI, Uvicorn, Pydantic, Streamlit, and Requests.
`run_backend.bat`	Windows backend launcher	Creates `.venv` if needed, installs requirements, and runs Uvicorn with reload enabled.
`run_frontend.bat`	Windows frontend launcher	Runs Streamlit using the project's `.venv`; the backend should already be running.
`.env.example`	Example environment settings	Shows a database path and a Gemini key placeholder, but these do not match the optional provider settings in the current outreach code.
`README.md`	Project setup and overview	Useful for quick-start commands, but some outreach statements are inconsistent with the service implementation; see Section 9.
`tests/`	Expected location for automated tests	No test source files were present when this guide was prepared.
`.vscode/settings.json`	VS Code workspace setting	Points the Python extension at `.venv\\Scripts\\python.exe`. The setup instructions create this environment; a different local virtual environment may need a setting change.
`.kilo/kilo.jsonc` and `.kilo/.gitignore`	Optional Kilo tooling metadata	Editor/agent-tool configuration and ignore patterns, not needed to run the SalesGenie application.
`.venv312/`, `__pycache__/`	Local Python environment and bytecode cache	Machine-generated development artifacts, not application source. They can be recreated; do not treat them as runtime code or share them unnecessarily.

3. Module 1 — Lead Management

This is the foundation of the application. It stores each prospect as a row in SQLite and supports creating, listing, searching, filtering, reading, updating, and deleting leads. All three other modules load a lead through this module before working on it.

Fields stored on a lead

Field	Meaning	Validation/default in create request
`id`	Unique database identifier	SQLite-generated integer primary key.
`company_name`	Company or account name	Required; 1–200 characters.
`industry`	Industry label used for hints	Defaults to `Unknown`; up to 100 characters.
`contact_name`	Person's name for display and outreach	Defaults to `there`; up to 150 characters.
`contact_title`	Contact's role or job title	Defaults to an empty string; up to 150 characters.
`email`, `phone`	Contact methods	Optional strings; email format is not validated by the current schema.
`company_size`	Approximate employee count	Non-negative integer; defaults to 0.
`annual_revenue`	Annual revenue value	Non-negative number; defaults to 0. No currency/unit is stored.
`engagement_score`	Engagement indicator	Number from 0 to 100; defaults to 0.
`website_visits`, `email_opens`, `email_clicks`	Activity counts	Non-negative integers; defaults to 0.
`budget_confirmed`	Whether budget is known to be confirmed	Boolean; defaults to false.
`need_identified`	Whether a business need has been identified	Boolean; defaults to false.
`decision_maker`	Whether the contact is marked as a decision-maker	Boolean; defaults to false.
`lead_status`	Pipeline stage	Defaults to `New`; create allows up to 50 characters.
`notes`	Free-text sales notes	Defaults to an empty string.
`created_at`, `updated_at`	Record timestamps	Set by SQLite; updates refresh `updated_at`.

What the user can do in the interface

- Add a lead with company, contact, industry, status, engagement and qualification information.
- Search by company, contact, email, or industry; filter by status; refresh the list.
- Edit the selected lead's status, engagement score, and notes.
- Delete a selected lead.

The API's PATCH model supports more fields than the current edit form exposes. The UI's table is a summary, not a complete display of every stored field.

4. Module 2 — Lead Intelligence

Lead Intelligence creates a short company summary, a broad potential-needs hint, and a list of suggested opportunities. It uses the lead's industry, engagement score, email clicks, budget flag, and identified-need flag.

Industry-to-needs hints

Industry recognized by helper	Potential needs returned
Technology	Automation, scalability, integration, and security
Healthcare	Workflow efficiency, privacy, and reliable operations
Finance	Risk control, compliance, reporting, and efficiency
Retail	Customer experience, inventory visibility, and sales growth
Manufacturing	Operational efficiency, downtime reduction, and process visibility
Education	Student engagement, administrative efficiency, and reporting
Other or unrecognized	Process efficiency, visibility, and measurable business outcomes

Matching is case-insensitive substring matching against the stored industry text. Opportunities are added when the engagement score is at least 60 or there is at least one email click; when budget is confirmed; and when a need is identified. If none of those conditions apply, the function recommends discovery questions about problem, urgency, budget, and buying process.

> **Interpretation:** These are generic suggestions generated from user-entered fields. They are not researched company facts, product recommendations, or verified business needs.

5. Module 3 — AI Outreach

The Outreach page lets the user choose a lead, select a tone (`professional`, `friendly`, or `concise`), and specify a goal such as scheduling a discovery call. The endpoint returns a subject, body, and note. It does not send email, save drafts, or connect to an email inbox.

Template behavior

Without a successful provider response, the service creates a draft from the contact name, company, industry, selected tone, and goal. The subject is a fixed “Exploring priorities” style subject. The body contains a greeting, a tone-specific opening, a generic industry statement, the requested goal, and a sign-off. The text should be checked and personalized by a human before use.

Optional provider behavior in the current source

The service also checks for an environment variable named `GROQ_API_KEY` and, if it exists, attempts a request to a Groq-compatible chat-completions endpoint. On failure, it silently falls back to the template. The code currently sends a fixed placeholder authorization value rather than reading the key into the request header, so

this branch is not correctly authenticated as written. The exception is swallowed, which can make provider failures hard to diagnose.

6. Module 4 — Lead Scoring

The scorer adds points when qualification conditions are met, sums the points, caps the result at 100, and returns the contributing factors, priority tier, conversion-probability note, and suggested next action.

Condition	Points
Engagement score is at least 70	25
One or more email clicks	10
At least two email opens	5
At least three website visits	10
Budget confirmed	15
Business need identified	15
Decision-maker identified	10
Company size is at least 100	5
Industry is not Unknown, blank, or Other	5

Score	Tier	Recommended action
0–39	Low	Gather missing information and use a low-pressure nurture message.
40–69	Medium	Share a relevant case study and ask one qualifying question.
70–100	High	Prioritize a personalized follow-up and propose a discovery call.

The maximum possible raw point total is 100 with these rules; the explicit cap protects the output if rules are later changed. The response deliberately sets `conversion_probability` to null. A score is a prioritization heuristic, not a statistically validated probability that a lead will convert.

7. API reference

Method and path	Purpose and behavior
`GET /health`	Returns an OK status and the four module identifiers.
`POST /leads`	Validates a lead, inserts it, and returns the stored record. Success status is 201.
`GET /leads`	Lists leads. Optional `search` matches company/contact/email/industry; `status` filters exact status; `limit` is from 1 to 500 and defaults to 100.
`GET /leads/{lead_id}`	Returns one lead, or HTTP 404 when it does not exist.

Method and path	Purpose and behavior
`PATCH /leads/{lead_id}`	Updates supplied fields and the updated timestamp. An empty update returns HTTP 400; missing lead returns 404.
`DELETE /leads/{lead_id}`	Deletes a lead; returns a confirmation or HTTP 404 if missing.
`GET /leads/{lead_id}/company-analysis`	Returns company summary, industry needs hint, opportunities, and a source note.
`GET /leads/{lead_id}/intelligence`	Returns company analysis and scoring together.
`GET /leads/{lead_id}/score`	Returns points, tier, factors, probability disclaimer, and recommendation.
`POST /leads/{lead_id}/outreach`	Accepts tone and goal and returns an email draft. Tone must be one of the three allowed values.

FastAPI generates interactive API documentation at `http://127.0.0.1:8000/docs` while the backend is running. It is useful for trying requests without the Streamlit pages.

8. Database and data files

The database layer reads `DATABASE_PATH`; if unset, it uses `./data/salesgenie.db` relative to the process's current working directory. The parent folder is created if necessary. At backend startup, `init_db()` runs a `CREATE TABLE IF NOT EXISTS` statement for `leads`. It does not perform schema migrations or import the CSV.

SQLite boolean flags are stored as integer values (0 or 1). The row conversion helper turns those three flags back into Python booleans in API responses. Queries use parameterized values. The table has created/updated timestamps and no separate user, account, audit-history, or email-draft tables.

`sample_leads.csv` contains three illustrative records and columns that map mostly to lead fields. It is a manual sample only: no upload/import screen or automatic seeding is implemented. In particular, the CSV does not include the annual revenue field.

9. Configuration, documentation differences, and practical caveats

Area	What the checked-in files indicate	Practical effect
Outreach provider	README says template-only/no paid LLM, while `services.py` attempts Groq when `GROQ_API_KEY` is set.	The source, not the README, determines runtime behavior. A configured provider may receive lead contact/company details in the prompt.
Provider authorization	The request header in the Groq branch is a fixed placeholder, not the environment key.	The provider call will not authenticate properly as written; failed calls silently fall back to the template.

Area	What the checked-in files indicate	Practical effect
Environment example	`.env.example` names `GEMINI_API_KEY`; the service checks `GROQ_API_KEY`. The app does not load a .env file itself.	Copying the example does not configure the optional service branch. Environment variables must be set in the process environment.
Sample import	README suggests adding test leads through the UI or API; UI text mentions importing samples through API, but there is no CSV import endpoint.	Create leads individually through the form or `POST /leads`; CSV upload is not available.
Validation	`EmailStr` is imported but the actual email fields are typed as optional strings.	Email format is not checked by these models. Several update fields also have fewer constraints than create fields.
Testing	No automated test source files were found in `tests/`.	There is no project test suite to run for regression coverage at present.

For any real provider integration, align the README and environment example with the code, send the secret through a secure environment variable, use it in the authorization header, avoid swallowing provider errors silently, and decide explicitly whether personal lead data may be sent to the provider. Do not commit API keys or real customer records.

10. Running the project on Windows

Option A — two PowerShell terminals

Open PowerShell in the project folder. Create the environment and install dependencies once:

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

In the first terminal, start the backend and leave it running:

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

In a second terminal in the same folder, start the frontend:

```
.\.venv\Scripts\python.exe -m streamlit run frontend.py
```

Open the Streamlit address printed in the terminal (normally `http://localhost:8501`). The API docs are at `http://127.0.0.1:8000/docs`.

Option B — batch files

Run `run_backend.bat` first. It creates `.venv` if absent, installs requirements, and starts Uvicorn. Then run `run_frontend.bat` in a second terminal/window. The batch files assume the project folder is the working directory resolved from the script location.

Backend address configuration

The frontend reads `SALESGENIE_API` and defaults to `http://127.0.0.1:8000`. Set it before launching Streamlit if the backend is hosted elsewhere. The frontend checks `/health` when it starts and stops with a warning if the API is unavailable.

11. Typical user workflow

1. Start the API and Streamlit frontend.
2. Open Lead Management and create a lead with accurate company, contact, activity, and qualification information.
3. Search or filter the lead list; update pipeline status and notes as new facts are learned.
4. Use Lead Intelligence to review generic industry needs and discovery prompts.
5. Use Lead Scoring to prioritize follow-up and inspect which input fields contributed points.
6. Use AI Outreach to create a draft; verify every statement and edit it before sending through a separate email system.

12. Scope and what is not implemented

- No conversation/transcript intelligence module.
- No analytics dashboard, charts, or pipeline reporting module.
- No machine-learning training or outcome-based conversion prediction.
- No external company research or validation of facts.
- No email sending, inbox integration, or persistent draft history.
- No CSV upload/import workflow despite the sample CSV file.
- No authentication, user roles, or multi-tenant access control is present in the described code.
- No automated test suite was present during inspection.

This guide explains the files and behavior in the project snapshot reviewed. It is an explanatory document, not a claim that the application is production-ready or that its inferred sales guidance is verified.